MetalNet GUI Profiler Presentation Guide

UI Architecture, Tech Stack, and Technicalities

Speaking Notes & Reference Guide

June 11, 2026

1 Introduction to the UI

The **MetalNet Profiler** is a real-time, multi-threaded training and inference dashboard. It acts as the visual nerve center of the project, allowing users to watch a Convolutional Neural Network (CNN) learn, inspect performance bottlenecks, and interactively tweak hardware and learning parameters.

2 The Tech Stack: What is Used and Why

The GUI is built using a lightweight, high-performance C++ stack designed to introduce zero garbage collection spikes and minimal CPU overhead:

- **Dear ImGui** (Immediate-Mode GUI):
  - *What it is:* A highly popular, bloat-free immediate-mode graphical user interface library for C++.
  - *Why it's used:* Traditional retained-mode GUIs (like Qt or JavaFX) require complex event loops, widget hierarchies, and high memory overhead. ImGui renders the UI dynamically on every frame, which is extremely efficient for real-time visualization of rapidly changing neural network statistics.
- **ImPlot**:
  - *What it is:* An immediate-mode plotting add-on built on top of Dear ImGui.
  - *Why it's used:* It handles high-frequency data streams (e.g., thousands of steps per minute) to render scrolling plots (Loss, Accuracy, and Throughput) without allocations or lag.
- **GLFW & OpenGL3**:
  - *What it is:* GLFW creates the operating system window, manages OpenGL graphics contexts, and handles keyboard/mouse input. OpenGL3 is the backend used to draw vector geometry on the GPU.

- *Why it's used:* This guarantees cross-platform compatibility (works on Windows, Linux, and macOS) and uses hardware acceleration to render the UI at 60 FPS.

3 Core UI Features & Architecture

3.1 The Dual-Threaded Model

To ensure the UI remains fully responsive (60 FPS) and never freezes during heavy training or inference cycles, the application uses two dedicated threads:

1. **Main UI Thread:** Runs the GLFW event loop and renders the ImGui panels.
2. **Background Worker Thread:** Dedicated to running the training steps or inference loops.

Technicality (Synchronization): The threads communicate using `std::atomic` flags for controls (alive, stop, thread count, steps) and a `std::mutex` to safely lock and snapshot statistics (vectors of loss, accuracy, and layer performance data) to prevent race conditions during UI updates.

3.2 Interactive Parameters Panel

Users can dynamically change configuration parameters on the fly:

- **Batch Size & Steps:** Adjust the training volume and length.
- **Threads (OpenMP):** A slider controlling the number of CPU cores utilized. Sliding it immediately adjusts the thread allocation in OpenMP for the convolution math.
- **Learning Rate & Weight Decay:** Modify optimizer hyperparameters interactively.

4 Advanced Visualizations (Technical Details)

4.1 Compute Heatmap

- **How it works:** Renders a scrolling 2D grid utilizing the *Plasma* colormap.
- **Technicality:** The vertical axis lists the layers of the network in execution order, and the horizontal axis shows chronological training steps. The color intensity represents the total execution time (in milliseconds) for that layer, making it trivial to spot performance bottlenecks (e.g., convolution taking 95% of the time compared to pooling).

4.2 Convolution Filter & Feature Map Visualization

This view exposes the internal weights and representations of the network:

- **Input Layer (Layer 0):** The first convolutional layer has 3 input channels (Red, Green, Blue). The weights are mapped directly to BGR values to display the learned kernels as color filters.

- **Deeper Layers:** Deeper layers have many channels and cannot be directly represented as color. Instead, the UI computes the **magnitude** (Euclidean norm) of the weights across channels and renders them in grayscale.
- **Feature Maps:** Displays live activation maps (what each filter "sees" on a sample input) in real time.

4.3 Perfetto Trace Exporter

With one click, the UI exports a Chrome-compatible JSON trace (`metalnet_trace.json`). This records the nanosecond-level start and end times of every layer pass, allowing developer inspection in Google Chrome's `chrome://tracing` or Perfetto UI.

5 Speaking Script / Quick Talking Points

Here is a step-by-step guide your friend can read or refer to during the presentation:

1. **Introduction:** "Hello everyone. I will be presenting the User Interface and Profiling capabilities of MetalNet. We didn't want this library to just be a black box; we wanted a dashboard to see the neural network learn and analyze performance in real time."

2. **The Stack:** "For the UI, we used a highly responsive C++ stack. It uses **Dear ImGui** and **ImPlot** for rendering the graphs, and **GLFW/OpenGL3** for GPU-accelerated window management. Because it is immediate-mode, it introduces zero memory overhead and runs at a smooth 60 frames per second."

3. **Thread Safety:** "Architecturally, the GUI is fully multi-threaded. The UI runs on the main thread, while all neural network training and inference happen on a background worker thread. They communicate safely via atomic flags and mutexes. This prevents the window from freezing, even when running the CPU at 100% load."

4. **Key Features & Heatmap:** "On the left sidebar, users can change parameters like batch size, optimizer learning rates, and the number of threads dynamically. On the main content pane, we have scrolling charts tracking Loss and Accuracy. The most notable component is the **Compute Heatmap**, which displays a scrolling color grid showing the execution time of each individual layer in real time, making bottlenecks immediately visible."

5. **Visualizing Weights:** "Lastly, we visualize the network's filters. The first layer is rendered in RGB colors because it has 3 channels. Deeper layers are represented as grayscale magnitudes. We also show live activation maps, demonstrating exactly what features the model is detecting as training progresses."